

Guide 1

Introduction to Big Data Processing with Apache Spark

2025/2026

Spark HowTo

Create the VM using the VagrantFile

```
vagrant up
```

Install Python version 3.12 and set up virtual environment

```
sudo apt-get install python3.12
sudo apt install python3.12-venv
python3 --version #confirm python version and if needed change python alternative
python3 -m venv env-spark
```

Install Java and add it to the `bashrc`

```
sudo apt install openjdk-17-jdk
echo "export JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-arm64" >> ~/.bashrc
echo "export PATH=$JAVA_HOME/bin:$PATH" >> ~/.bashrc
source ~/.bashrc
```

Install PySpark and Jupyter Notebooks

```
source env-spark/bin/activate
pip install pyspark notebook
```

1 Intro to Apache Spark

Consider the IMDB datasets schemas available at <https://developer.imdb.com/non-commercial-datasets/> and downloadable at <https://datasets.imdbws.com/>.

As these datasets are in TSV (*tab-separated values*) data format, you should define the read function to read this specific type of data, for instance:

```
titleData = spark.read.format("csv")
    .option("inferSchema", "true")
    .option("sep", "\t")
    .option("header", "true")
    .load(titleFile)
    .cache()
```

1.1 Explore processing data in Spark by combining several datasets.

1. Compute the number of titles per `titleType`.
2. Which years have the most movies and series registered on IMDB?
3. Which director creates the longest movies, and what is the average duration of its movies (in hours)?
4. Are there any records without a year? How many and which ones?
5. Which titles have the highest ranking?
6. Which category of job is the most performed by artists?
7. Which actors have appeared in the most highly-rated movies?
8. Which titles have the most votes?

Note: Check the appendix for some hints on how to use Spark to answer the previous questions.

1.2 Spark can also allow us to query and finally save the output to other data formats.

1. Using the DataFrame function `write.parquet(out_folder)`¹, export the `titles` DataFrame to Parquet.
2. Export the DataFrame again, using `gzip` compression, by providing the `compression='gzip'` parameter to `write.parquet`.
3. Export the DataFrame again, partitioned by the `startYear` column (`partitionBy='startYear'`).
4. Using the previous export and the function `spark.read.parquet(in_folder)`, compute the number of titles released in 2024.

2 Questions

1. How long does it take to perform the several queries on top of each dataset?
2. What happens when you call `.cache()` on a DataFrame? How can you check if data is actually cached?
3. What is the difference between `.cache()` and `.persist()`?
4. What happens if a dataset is too big to fit in memory? How does Spark handle this?

3 Learning Outcomes

Get familiarized with the Spark framework and with data processing functions.

¹<https://spark.apache.org/docs/3.5.1/sql-data-sources-parquet.html>

A Hints and Useful Spark Functions

For further details on Apache Spark functionalities, refer to the official documentation: <https://spark.apache.org/docs/latest/api/python/reference/index.html>

A.1 Reading and Exploring Data

Read TSV files with tab separation and headers:

```
df = spark.read.format("csv")
    .option("sep", "\t")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("file_path")
```

Display the schema of a DataFrame:

```
df.printSchema()
```

Show the first rows of a DataFrame:

```
df.show(5)
```

A.2 Processing Data

Count distinct values:

```
df.groupBy("titleType").count().show()
```

Filter data:

```
df.filter(df.startYear.isNull()).show()
```

Aggregate functions:

```
from pyspark.sql.functions import avg, max, min, count
```

```
df.groupBy("director")
    .agg( max("runtimeMinutes")
    .alias("max_runtime"), avg("runtimeMinutes")
    .alias("avg_runtime") )
    .show()
```

Sort data:

```
df.orderBy(df.numVotes.desc()).show(10)
```

Join datasets:

```
merged_df = df1.join(df2, "common_column", "inner")
```

A.3 Saving and Exporting Data

Save as Parquet:

```
df.write.parquet("output_folder")
```

Save with compression:

```
df.write.option("compression", "gzip").parquet("output_folder")
```

Save partitioned by column:

```
df.write.partitionBy("startYear").parquet("output_folder")
```

Read from Parquet:

```
spark.read.parquet("output_folder")
```

A.4 Optimizations and Caching

Cache a DataFrame in memory:

```
df.cache() df.count() # Triggers caching
```

Persist with different storage levels:

```
from pyspark.storagelevel import StorageLevel
```

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

Check if data is cached:

```
df.is_cached
```